Guarded Consumers

Projection, Evaluation and Ordered Effects

Mettapedia Working Notes

31 October 2026

Abstract

Six Lean modules extend the stored-ID and named-shape-store theory with contracts for guard compilation, binding projection and effect batching. A short-circuit guard compiler preserves results and executed read certificates. A separate bridge compares it with eager Boolean relations only for pure, complete, singleton-Boolean operands. Projecting bindings retains guards, effects and answer templates together, preserving complete execution even on partial failure. Ordered insert-on-absence plans preserve authored effects; a narrower commutative set fragment admits multiplicity summaries with exact branch-sensitive answer counts. These are logical permission laws, not native compiler extraction or runtime performance guarantees.

1 Two evaluation regimes must not be confused

The guard language has literals, resolved query atoms, negation, conjunction and disjunction. A query returns one of a Boolean answer, a fault, or an unavailable realization. The last case denotes an optimization boundary, not source-level falsity or a new source error. A source spelling does not acquire a Boolean meaning merely because a compiler recognizes its name.

`GuardConjunctionCompiler` independently interprets the short-circuit language and compiles it to the shared `ReadOnlyQuery.Program` interface. The proof preserves the answer and complete ordered read certificate. Constant-controlled branches and double negations normalize without changing that observation. A validated executed-read certificate permits reuse in a new world, but only for the same admitted program and authority context. Changes of definitions, profile or primitive interpretation are not licensed by a binding-only key.

The inspected PeTTa Boolean-relation path evaluates its argument tuple before using a truth table. It must therefore not be identified with this short-circuit language without another proof. For a pure finite-answer source, an eager conjunction is the Cartesian combination

$$E(g \wedge h) = [\, a \wedge b \mid a \in E(g),\ b \in E(h) \,],$$

where occurrences and order are retained. `GuardEvaluationRegimes` proves equality of eager and short-circuit *answers* when every atom produces exactly one Boolean. This bridge does not preserve all reads: a false left operand can make the right query unnecessary.

Three controls delimit the qualification:

- an eager false-and-empty expression has no answers, not one false answer;
- a false left answer and two right answers produce two false occurrences;
- a fault or effect on the right cannot be erased just by Boolean algebra.

The first two are kernel-checked list-semantic counterexamples. A small runtime control also observed right-operand effects, zero answers for an empty right operand, and duplicate false answers on the inspected saved build. General source extraction must additionally establish purity, termination, absence of binding refinements and the active primitive/profile interpretation.

2 Projection retains the complete consumer

`GuardedConsumerProjection` represents literal or variable operands, guard calls, ordered effects, and finite answer templates. Its support is computed structurally:

$$\begin{aligned} S(\text{answer}(T)) &= S(T), \\ S(\text{effect}(e, a, p)) &= S(a) \cup S(p), \\ S(\text{branch}(g, p, q)) &= S(g) \cup S(p) \cup S(q). \end{aligned}$$

The implementation keeps lists of coordinates; repeated coordinates are allowed. A finite capture stores their binding values. Restoring arbitrary fallback values elsewhere preserves the consumer's complete result: answer occurrences, final world, ordered query reads, ordered effect calls and completion status. An effect can leave a partially changed world and then fault; that state is preserved too.

This closes the gap between "unused by the answer" and "unused by the consumer." A variable read only by a guard is still necessary. Likewise, equal returned answer lists do not justify dropping an effect argument. The static support is conservative rather than asserted minimal: a separately certified specialization can eliminate unreachable branches first.

Projection is not memoization. Even a consumer with empty binding support may observe a mutable world. A negative control changes a zero-argument membership query while retaining the same binding capture, changing the answer. Reuse across worlds needs read validation, and reuse across program or profile changes needs the corresponding authority checks.

3 Ordered plans for insert-on-absence guards

For a guard key k, the fresh branch runs when k is absent. The required locality contract is: on successful completion it makes k present and leaves the other guard keys unchanged. Other world effects may be arbitrary and noncommutative. On failure its actual partial state is retained.

`OrderedInsertGuardBatch.plan` walks the candidates without performing world effects, recording fresh or present tokens against a private simulated key set. It records every present token too, because the other branch can produce an answer. `execute` then runs these tokens in the original order. The theorem `planned_refines` equates it with independent sequential guard evaluation, including failures, unavailable outcomes, answer multiplicity, ordered effects and final state.

The planner's final key set is not bulk-published. If an early branch fails, later tokens do not execute and their keys do not become visible. The proof allows a failed branch to have made partial changes, rather than silently rolling them back. Batches can be split at any candidate boundary under the same sequential order; reordering arbitrary effects is not justified.

The key must encode all consumer information needed for these branch functions. The projection theorem provides a finite binding capture, but does not by itself justify treating two rows as the same guarded action. Locality of guard-set changes and the complete effect contract remain separate admission conditions.

`FirstRepresentativeGuardBatch` separates the guard key from the complete consumer row. Rows with the same guard key may carry different effect arguments. An ordered plan retains the first actual row for the fresh branch and each later row for its present branch, and still refines sequential execution. This allows effect variables outside the guard key when enumeration and occurrence identity are preserved. It does not permit an arbitrary representative: reversing two rows with the same key but different payloads can change the result.

4 A wider commutative region with exact multiplicity

For set-valued insertions and a count-only observer, stronger permission is available. `GuardedSetMultiplicity` models a state consisting of guard keys, a set of facts and an answer count. A fresh key inserts a fixed set of payloads; a present key changes neither set. Each branch has its own answer count. Steps commute in this fragment, and permutation of candidates preserves the full modeled state.

For multiplicity $m > 0$, let $f(k)$ and $p(k)$ be the fresh-branch and present-branch answer counts. The exact contribution is

$$\begin{cases} m\,p(k), & k \text{ initially present,} \\ f(k) + (m-1)\,p(k), & k \text{ initially absent.} \end{cases}$$

Zero multiplicity changes nothing. The formula is proved against repeated one-at-a-time execution. A reference checker validates that grouped counts expand to exactly the source occurrence bag; checked groups then preserve the final state. Missing a duplicate is rejected.

This permits an empty present branch: with $f(k) = 1$, $p(k) = 0$, one hundred occurrences of a new key produce one answer, whereas an already held key produces none. It also preserves multiple answers within the fresh branch. It does not deduplicate branch-internal effects or answers.

This region excludes ordered answer streams, arbitrary stateful effects, removal, primitive faults and intermediate observers. Those need the ordered model or additional laws. Natural-number arithmetic is used; native overflow and saturation policies are not specified by the theorem. The grouping checker is a logical reference, not a claim that the production grouping algorithm has been verified.

5 Integration and qualification

The six modules under `Mettapedia/Machines/` are `GuardConjunctionCompiler`, `GuardEvaluationRegimes`, `GuardedConsumerProjection`, `OrderedInsertGuardBatch` and `GuardedSetMultiplicity` and `FirstRepresentativeGuardBatch`. `GuardOptimizationAudit` inventories module-owned mathematical bodies and their transitive axioms. Standard `propext`, `Classical.choice` and `Quot.sound` are permitted; admissions and additional axioms are not. Compiler runtime counterparts are listed separately from mathematical bodies.

The immediate native use is a specification for combining filters with existence guards, retaining all consumer variables, and counting grouped guarded insertions correctly. It supplies no new surface builtin and no native optimization by itself. The frontend must establish source/profile authority, pure scalar operands and stable captured bindings. Mutation validation and publication must respect the selected observer, and implementation-level resource/refusal behavior must remain correct.

These results extend the contracts described in `NamedShapeStores.tex` and `StoredIdUnificat`

`ion.tex`. Restricted transducers and CTL fixed points, canonical decision-diagram operations, higher derivatives and optimal cuts remain separate parts of the wider theory.